1. Mục tiêu

- Hiểu rõ khái niệm Docker, Dockerfile, image và container, và vì sao đây là kỹ năng bắt buộc khi phát triển và triển khai ứng dụng phía server (đảm bảo môi trường chạy nhất quán giữa máy dev và server thật).
- Có khả năng viết Dockerfile để đóng gói (containerize) chính ứng dụng backend/API mà nhóm đang phát triển trong học phần — không phải một ví dụ "Hello World" tách biệt.
- Thực hiện được các lệnh Docker cơ bản để build image, chạy container, quản lý container, và kết nối container với các thành phần khác (ví dụ database) mà ứng dụng cần.
- Biết cách tối ưu Dockerfile (multi-stage build, `.dockerignore`) và cấu hình container chạy với user không phải root, phù hợp thực hành bảo mật đã học ở buổi Linux.
- Chuẩn bị tốt cho phần thuyết trình và demo các thao tác với Docker trên ứng dụng thực tế.

2. Nội dung cần nghiên cứu và chuẩn bị

2.1. Giới thiệu Docker

- **Docker là gì?** Nền tảng mở để phát triển, vận chuyển và chạy ứng dụng. Sử dụng công nghệ ảo hóa cấp hệ điều hành (containerization).
- **Lợi ích của Docker:** Cô lập môi trường, nhất quán giữa các môi trường (dev, test, prod), dễ dàng triển khai và mở rộng.
- **Khái niệm cơ bản:**
  - **Image:** Một template chỉ đọc chứa các hướng dẫn để tạo ra một container. Giống như một bản thiết kế.
  - **Container:** Một instance có thể chạy được của một image. Giống như một ngôi nhà được xây từ bản thiết kế.
  - **Dockerfile:** Một file văn bản chứa các lệnh để tự động build một Docker image.

2.2. Dockerfile

- **Cấu trúc cơ bản của Dockerfile:**
  - `FROM`: Chỉ định base image.
  - `WORKDIR`: Thiết lập thư mục làm việc.
  - `COPY`: Sao chép file/thư mục từ host vào image.
  - `RUN`: Thực thi lệnh trong quá trình build image.
  - `EXPOSE`: Khai báo cổng mà container sẽ lắng nghe.
  - `CMD`: Lệnh mặc định sẽ chạy khi container khởi động.
  - `ENTRYPOINT`: Lệnh sẽ chạy khi container khởi động (thường dùng để cấu hình).
- **Viết Dockerfile cho ứng dụng backend:** Áp dụng trực tiếp lên ứng dụng backend/API mà nhóm đang phát triển trong học phần (Node.js, .NET Core, Python...).
  - Đảm bảo ứng dụng có thể chạy độc lập trong container.
  - **`.dockerignore`:** Loại trừ các file/thư mục không cần thiết khi build (ví dụ: `node_modules`, `bin/`, `obj/`, `.git`, file `.env` chứa secret) để image nhỏ gọn và an toàn hơn.
  - **Multi-stage build:** Tách stage build (compile/publish) và stage runtime để giảm kích thước image cuối cùng, không mang theo SDK/công cụ build không cần thiết khi chạy production.
  - **Chạy container với user không phải root:** Tạo user riêng trong Dockerfile (liên hệ lại kiến thức phân quyền ở buổi Linux) để hạn chế rủi ro bảo mật khi ứng dụng bị khai thác lỗ hổng.

- **Biến môi trường (`ENV`, `ARG`):** Cấu hình connection string, port, hoặc các thông số khác của ứng dụng qua biến môi trường thay vì hard-code, để dễ dàng đổi cấu hình giữa các môi trường (dev/test/prod).

2.3. Các lệnh Docker cơ bản

- `docker build -t <image_name> .`: Build một Docker image từ Dockerfile trong thư mục hiện tại.
- `docker images`: Liệt kê các Docker image có trên hệ thống.
- `docker run -p <host_port>:<container_port> --name <container_name> <image_name>`: Chạy một container từ image, ánh xạ cổng, đặt tên cho container.
- `docker ps`: Liệt kê các container đang chạy.
- `docker ps -a`: Liệt kê tất cả các container (kể cả đã dừng).
- `docker exec -it <container_name> <command>`: Thực thi lệnh bên trong một container đang chạy (ví dụ: `docker exec -it my_app_container bash`).
- `docker logs <container_name>`: Xem log của container.
- `docker stop <container_name>`: Dừng một container đang chạy.
- `docker rm <container_name>`: Xóa một container (phải dừng trước khi xóa).
- `docker rmi <image_name>`: Xóa một image.

3. Chuẩn bị thuyết trình và demo

- **Slide:** Giới thiệu Docker, các khái niệm, cấu trúc Dockerfile và các lệnh cơ bản.
- **Bắt buộc — gắn với ứng dụng backend thực tế:** Không dùng ví dụ "Hello World" tách biệt; toàn bộ demo phải thực hiện trên chính ứng dụng backend/API của nhóm:
  - Viết Dockerfile để đóng gói ứng dụng backend của nhóm, có sử dụng multi-stage build và `.dockerignore`.
  - Cấu hình container chạy bằng user không phải root.
  - Build image từ Dockerfile đó (`docker build -t <tên_nhóm>-api:v1 .`), gắn tag rõ ràng.
  - Chạy container từ image vừa build, ánh xạ cổng, truyền biến môi trường (ví dụ connection string tới database) và kiểm tra ứng dụng hoạt động bằng cách gọi thử một API thật (ví dụ qua `curl` hoặc Postman).
  - Sử dụng các lệnh `docker ps`, `docker logs`, `docker exec` để tương tác, kiểm tra và debug container khi cần.
  - Dừng và xóa container, sau đó xóa image.
- **Thời gian:** Phân bổ thời gian hợp lý cho lý thuyết và demo.

4. Phân công công việc và ước lượng thời gian (Gợi ý)

Task	Người thực hiện (Gợi ý)	Ước lượng (Point)
Nghiên cứu về Docker, Image, Container	TV1	3
Nghiên cứu cấu trúc Dockerfile và các lệnh	TV2	3
Chuẩn bị nội dung slide thuyết trình	Nhóm trưởng	2
Viết Dockerfile (multi-stage, non-root user, `.dockerignore`) cho ứng dụng backend của nhóm	TV3	5

Task	Người thực hiện (Gợi ý)	Ước lượng (Point)
Thực hành các lệnh Docker cơ bản và chuẩn bị demo	TV1, TV2	4
Tổng hợp, kiểm tra và chạy thử demo	Cả nhóm	2
Luyện tập thuyết trình và demo	Cả nhóm	1
Tổng cộng		**20**

5. Tài liệu tham khảo

- **Tài liệu chính thức:** Docker Documentation (docs.docker.com).
- **Bài viết:** Các bài hướng dẫn Docker cho người mới bắt đầu.
- **Video:** Các khóa học hoặc video hướng dẫn Docker cơ bản trên YouTube.

6. Ghi chú quan trọng

- **Thực hành là cách tốt nhất để học Docker.**
- **Hiểu rõ sự khác biệt giữa Image và Container.**
- **Luôn kiểm tra log khi có lỗi.**
- **Docker hóa ứng dụng thật, không phải ví dụ mẫu:** Mục tiêu của buổi này là để nhóm có thể tự đóng gói ứng dụng backend đang phát triển và chạy được nó ở bất kỳ máy nào có Docker — đây là kỹ năng nền tảng trước khi học Docker Compose ở buổi tiếp theo.

Chúc các bạn học tập hiệu quả và thành công!